

Interviewer Guide & Evaluation Reference

Solution Code · Scoring Rubric · Q&A Answers · Red & Green Flags

 Interviewer Only

Do Not Share with Candidates

Covers all 4 stages



How to Use This Guide

This document is structured to support both an **async code review** (before you meet the candidate) and a **live technical discussion**. Use Part 2 (solutions) to evaluate the submitted code quickly. Use Part 3 (Q&A) during the conversation to probe depth.

- ☐ Read the solution reference before reviewing the candidate's code
- ☐ Mark the scorecard as you review (last page)
- ☐ Flag interesting choices to discuss live — not everything is a mistake
- ☐ Pick 4–6 Q&A questions based on what they implemented
- ☐ Check the "red flags" section for patterns that need probing

Pre-Interview Checklist

- ☐ Run `Store.API` on `:5000` — confirm it responds
- ☐ Check that `scraper.db` exists and has rows
- ☐ Run `UltraStore.API` on `:5100` — hit `/api/products`
- ☐ Run `npm run dev` in `ultrastore-client`
- ☐ Note which tasks were completed / skipped / incomplete

Suggested 60-Minute Interview Structure

0–5 min

Setup & Orientation

Candidate runs the services. Confirm everything is working. Note any parts they skipped — that is fine and expected.

5–20 min

Code Walkthrough — Candidate Leads

Ask the candidate to walk you through their implementation. Listen for: architecture choices, trade-offs they considered, things they'd change with more time. **Don't correct — just listen and note.**

20–35 min

Targeted Code Review — You Lead

Focus on 2–3 specific areas: SQL injection safety, React Query cache keys, validation responses. Use the solutions in Part 2 to guide questions. Ask "why" before telling them what's wrong.

35–55 min

Technical Discussion

Pick 4–6 questions from Part 3. Prefer questions that connect to their actual implementation. Senior probes: scalability, trade-off reasoning. Junior probes: core correctness.

55–60 min

Candidate Questions + Wrap-Up

Leave time for candidate questions — how they engage with the role is signal too.



Level Expectations

● Junior (0–2 yr)

- Scraper runs but may not paginate
- SQL filtering works; may use string concatenation (flag this)
- Validation present but incomplete
- React Query hooks implemented; queryKey may be too broad
- Detail page renders basic info
- Stage 4: skips or attempts debounce/context but incomplete; limited scalability awareness

● Mid (2–5 yr)

- Scraper paginates, handles empty pages cleanly
- Parameterized SQL, proper WHERE clause
- All params validated; descriptive error messages
- Correct queryKey with all filter deps
- Understands useQuery vs useMutation
- Can explain offset vs cursor pagination
- Stage 4: completes at least one bonus (debounce or context); can discuss indexing

● Senior (5+ yr)

- Considers retry, backoff, idempotency in scraper
- Returns ProblemDetails (RFC 7807) for errors
- Context API integrated cleanly; aware of React Query staleTime strategy
- Has opinions on UPSERT key selection
- Stage 4: completes multiple bonuses; proactively covers scalability, indexes, and architectural trade-offs unprompted



Evaluation Rubric

Use the scorecard on the last page to record marks

AREA	WHAT EARNS FULL MARKS	PARTIAL CREDIT	MAX PTS
Scraper — basic	Fetches all products with pagination loop; terminates correctly when no more pages; assigns Guid IDs; saves with UpsertProductAsync	Fetches first page only, or hardcodes the page count	15
Scraper — quality	Logs progress; handles HTTP errors (non-2xx); DTOs defined cleanly; uses UTC timestamps	Works but no error handling; timestamps use local time	5
Backend — filtering	Dynamic WHERE clause with DynamicParameters (no string concatenation); all 4 filters (search, category, minPrice, maxPrice) work correctly; LIMIT/OFFSET correct; COUNT query runs	Some filters work; pagination off by one; string concat used	20
Backend — validation	All 4 params validated; each returns 400 with a clear message; minPrice > maxPrice caught; never returns 500 for bad input	Only some params validated; generic error message	10
React Query hooks	All 3 hooks implemented with Axios; queryKey includes all filter deps for useProducts; enabled flag on useProduct; error states handled in UI	Hooks work but queryKey is too broad (e.g. ['products'] only)	20
Products page	All filters wired up; filter change resets page to 1; pagination works; cards navigate to detail page	Search or category works but not price range; pagination off	10
Detail page	Route works; all fields rendered; image fallback; loading and error states handled; back navigation	Some fields missing; no loading state; no error state	10
Code quality	Consistent naming; no commented-out code; no security issues; TypeScript types correct; no unnecessary re-renders	Minor issues; some any casts; occasional code smell	5
Stage 4 — Debounce	useRef + setTimeout pattern; clearTimeout on each change; fires only after user stops typing	Attempts debounce but incorrectly (e.g. no clearTimeout)	+5
Stage 4 — Context API	ProductProvider implemented; useProductFilters hook works; App.tsx wrapped; no prop drilling in ProductsPage	Context created but not fully integrated	+5
Stage 4 — DB Index	CREATE INDEX added in InitializeAsync; correct column choice; can explain why	Knows about indexes conceptually but didn't add one	+5
Total (passing threshold ≈ 65 / 95)			95 + 15



i The Store.Scraper project includes **Microsoft.Extensions.Http**, so `System.Net.Http.Json` and `GetFromJsonAsync` are available without additional packages. Candidates who use a raw `HttpClient` with `JsonSerializer` are also correct.

ScraperService.cs – ScrapeAsync()

C#

```
public override async Task ScrapeAsync()
{
    using var http = new HttpClient();
    int page = 1, saved = 0;
    const int pageSize = 100;

    while (true) // ✓ paginated loop
    {
        var result = await http.GetFromJsonAsync<PagedDto>(
            $"http://localhost:5000/api/products?page={page}&pageSize={pageSize}");

        if (result is null || result.Items.Count == 0) break; // ✓ clean exit

        foreach (var item in result.Items)
        {
            await Repository.UpsertProductAsync(new Product
            {
                Id = Guid.NewGuid(), // ✓ fresh GUID
                Name = item.Name,
                Description = item.Description,
                Price = item.Price,
                ImageUrl = item.ImageUrl,
                CategoryName = item.CategoryName,
                AvailableAmount = item.AvailableAmount,
                CreatedAt = DateTime.UtcNow, // ✓ UTC
                UpdatedAt = DateTime.UtcNow,
            });
            saved++;
        }

        Logger.LogInformation("Page {Page}: {Count} products saved", page, saved);
        if (!result.HasNextPage) break; // ✓ uses API signal
        page++;
    }
}

// DTOs – GetFromJsonAsync uses JsonSerializerDefaults.Web (case-insensitive)
private record PagedDto(List<ItemDto> Items, bool HasNextPage);
private record ItemDto(string Name, string Description, decimal Price,
    string ImageUrl, string CategoryName, int? AvailableAmount);
```

REQUIRED Pagination loop — must call page 1, 2, 3 ... until HasNextPage is false or items is empty. Fetching only page 1 loses ~990 products.

REQUIRED Guid.NewGuid() for each product — the scraper owns the ID, not the Store.API (which uses int IDs).

DISCUSS Candidate uses local DTOs — ask: "Why not reuse the Store.API models directly?" (Separate concerns; scraper shouldn't depend on a different project.)

COMMON ISSUE Using DateTime.Now instead of DateTime.UtcNow. Ask: "What problems does local time cause in a distributed or cloud system?" (timezone shifts, ordering issues, DST bugs).

```
async scrape(): Promise<void> {
  let page = 1, saved = 0;
  const pageSize = 100;

  while (true) {
    const res = await fetch(
      `http://localhost:5000/api/products?page=${page}&pageSize=${pageSize}`);
    if (!res.ok) throw new Error(`Store.API returned ${res.status}`); // ✓ error check

    const data: PagedResult = await res.json();
    if (!data.items.length) break;

    for (const item of data.items) {
      await this.repository.upsertProduct({
        id:          crypto.randomUUID(),           // ✓ built-in, no library
        name:        item.name,
        description:  item.description,
        price:       item.price,
        imageUrl:    item.imageUrl,
        categoryName: item.categoryName,
        availableAmount: item.availableAmount,
        createdAt:   new Date().toISOString(),
        updatedAt:   new Date().toISOString(),
      });
      saved++;
    }
    console.log(`Page ${page}: ${saved} saved`);
    if (!data.hasNextPage) break;
    page++;
  }
}
```



ProductRepository.cs – GetProductsAsync()

C#

```
public async Task<PagedResult<Product>> GetProductsAsync(
    string? search, string? category, decimal? minPrice, decimal? maxPrice,
    int page, int pageSize)
{
    using var conn = CreateConnection();
    var conditions = new List<string>();
    var p = new DynamicParameters(); // ✓ parameterized – no injection

    if (!string.IsNullOrEmpty(search)) {
        conditions.Add("(Name LIKE @search OR Description LIKE @search)");
        p.Add("search", $"{search}%"); // ✓ wildcard on both sides
    }
    if (!string.IsNullOrEmpty(category)) {
        conditions.Add("LOWER(CategoryName) = LOWER(@category)"); // ✓ case-insensitive exact
        p.Add("category", category);
    }
    if (minPrice.HasValue) { conditions.Add("Price ≥ @minPrice"); p.Add("minPrice", minPrice); }
    if (maxPrice.HasValue) { conditions.Add("Price ≤ @maxPrice"); p.Add("maxPrice", maxPrice); }

    var where = conditions.Count > 0
        ? "WHERE " + string.Join(" AND ", conditions) : "";

    var total = await conn.ExecuteScalarAsync<int>( // ✓ COUNT with same WHERE
        $"SELECT COUNT(*) FROM Products {where}", p);

    p.Add("limit", pageSize);
    p.Add("offset", (page - 1) * pageSize); // ✓ correct OFFSET formula

    var items = await conn.QueryAsync<Product>(
        $"SELECT * FROM Products {where} ORDER BY Name LIMIT @limit OFFSET @offset", p);

    return new PagedResult<Product> {
        Items = items, TotalCount = total, Page = page, PageSize = pageSize
    };
}
```

CRITICAL **DynamicParameters is mandatory.** Any candidate who builds the WHERE clause via string interpolation like `"WHERE Name LIKE '%{search}%'"` has a SQL injection vulnerability — flag immediately.

DISCUSS LOWER() on both sides for category matching. Ask if they considered a collation setting instead — valid alternative, but LOWER() is explicit and portable.

NOTE The COUNT query and items query share the same DynamicParameters object — both must have the same WHERE clause. A common bug is running COUNT(*) without the WHERE clause, returning the wrong total.

```
public async Task<IActionResult> GetProducts(
    [FromQuery] string? search, [FromQuery] string? category,
    [FromQuery] decimal? minPrice, [FromQuery] decimal? maxPrice,
    [FromQuery] int page = 1, [FromQuery] int pageSize = 20)
{
    if (page < 1)
        return BadRequest("page must be ≥ 1."); // ✓ 400 with message
    if (pageSize < 1 || pageSize > 100)
        return BadRequest("pageSize must be between 1 and 100.");
    if (minPrice.HasValue && minPrice.Value < 0)
        return BadRequest("minPrice must be ≥ 0.");
    if (maxPrice.HasValue && maxPrice.Value < 0)
        return BadRequest("maxPrice must be ≥ 0.");
    if (minPrice.HasValue && maxPrice.HasValue && minPrice.Value > maxPrice.Value)
        return BadRequest("minPrice must not exceed maxPrice."); // ✓ cross-field check

    var result = await repo.GetProductsAsync(search, category, minPrice, maxPrice, page, pageSize);
    return Ok(result);
}

// Senior upgrade: return ProblemDetails for RFC 7807 compliance
// return Problem(detail: "pageSize must be between 1 and 100.", statusCode: 400);
```

GOOD Cross-field validation ($\text{minPrice} \leq \text{maxPrice}$) requires checking both values together — this is the most commonly missed case.

SENIOR PROBE Ask: "Should validation live in the controller or the repository?" (Controller — fail fast before touching the DB. Repository should trust its callers.)

SENIOR UPGRADE Using `Problem()` returns a `ProblemDetails` response (RFC 7807) — a more standard error shape. Not required, but shows API design awareness.



api/products.ts – all three hooks

TypeScript

```
export function useProducts(filters: ProductFilters) {
  return useQuery({
    queryKey: ['products', filters], // ✓ full filters object in key
    queryFn: async () => {
      const params = new URLSearchParams();
      if (filters.search) params.set('search', filters.search);
      if (filters.category) params.set('category', filters.category);
      if (filters.minPrice !== null) params.set('minPrice', String(filters.minPrice));
      if (filters.maxPrice !== null) params.set('maxPrice', String(filters.maxPrice));
      params.set('page', String(filters.page ?? 1));
      params.set('pageSize', String(filters.pageSize ?? 20));
      const { data } = await apiClient.get<PagedResult<Product>>(`/products?${params}`);
      return data;
    },
  });
}

export function useProduct(id: string | undefined) {
  return useQuery({
    queryKey: ['product', id],
    queryFn: async () => {
      const { data } = await apiClient.get<Product>(`/products/${id}`);
      return data;
    },
    enabled: !!id, // ✓ guard against undefined id
  });
}

export function useCategories() {
  return useQuery({
    queryKey: ['categories'],
    queryFn: async () => {
      const { data } = await apiClient.get<string[]>(`/categories`);
      return data;
    },
    staleTime: 5 * 60 * 1000, // ✓ 5 min – categories rarely change
  });
}
```

CRITICAL **queryKey must include all filter values.** A key of ['products'] means React Query serves stale data when filters change — the most common mistake.

REQUIRED enabled: !!id on useProduct prevents a request being fired with undefined as the path param when the component first mounts.

DISCUSS staleTime on useCategories — ask: "What does staleTime do?" (Tells React Query not to re-fetch in the background if the data is younger than N ms. Categories don't change often, so 5 min saves network traffic.)

```

const [search, setSearch] = useState('');
const [debouncedSearch, setDebouncedSearch] = useState(''); // ✓ separate state for query
const searchTimeout = useRef<ReturnType<typeof setTimeout> | null>(null);

function handleSearchChange(value: string) {
  setSearch(value); // ✓ input updates immediately
  if (searchTimeout.current) clearTimeout(searchTimeout.current); // ✓ reset on each key
  searchTimeout.current = setTimeout(() => {
    setDebouncedSearch(value); // ✓ query fires after 350ms idle
    setPage(1);
  }, 350);
}

// Use debouncedSearch (not search) in useProducts
const { data, isLoading, isError } = useProducts({
  search: debouncedSearch, // ✓ controlled – no request until user pauses
  category, minPrice: ..., maxPrice: ..., page, pageSize: 20,
});

```

KEY PATTERN Two pieces of state: search (input display) and debouncedSearch (query param). The input updates instantly; the API call waits. Without this split, the input lags visually.

COMMON MISTAKE Not clearing the timeout on each keystroke — the timer resets every key press, so the API fires 350ms after the *last* key, not each key.

```

export default function ProductDetailPage() {
  const { id } = useParams<{ id: string }>();
  const navigate = useNavigate();
  const { data: product, isLoading, isError } = useProduct(id);

  if (isLoading) return <div className="loading">Loading... </div>;
  if (isError || !product) return (
    <div><p className="error-banner">Product not found.</p>
    <button onClick={() => navigate('/')}>← Back</button></div>;
  )

  const inStock = product.availableAmount === null || product.availableAmount > 0;

  return (
    <div className="product-detail">
      <button onClick={() => navigate(-1)}>← Back</button> {/* ✓ history.back() */}
      <div className="product-detail-content">
        <img src={product.imageUrl} alt={product.name}
          onError={e => (e.target as HTMLImageElement).src = '...fallback...' } />
        <div>
          <span className="product-category">{product.categoryName}</span>
          <h1>{product.name}</h1>
          <p className="product-detail-price">${product.price.toFixed(2)}</p>
          <span className={`product-stock ${inStock ? 'in-stock' : 'out-of-stock'}`}>
            {inStock ? ` ${product.availableAmount ?? ''} in stock` : 'Out of stock'}
          </span>
          <p>{product.description}</p>
        </div>
      </div>
    </div>);
}

```



1 What is the difference between UUID and UUIDv7?

Classic UUIDs (v4) are entirely random — no ordering semantics. UUIDv7 embeds a millisecond-precision Unix timestamp in the high bits, making them monotonically increasing. This matters for B-tree indexes: random v4 UUIDs cause page splits on every insert; v7 inserts cheaply at the leaf edge like an auto-increment integer.

JUNIOR

"UUID is a unique ID, v7 adds a timestamp so they sort better."

MID

Mentions B-tree fragmentation, knows .NET 9 has `Guid.CreateVersion7()` or can use a library.

SENIOR

Connects it to write amplification in clustered indexes; mentions that the scraper scenario (bulk upsert) is exactly where random GUIDs hurt.

2 What is the difference between offset and cursor pagination?

Offset (`LIMIT n OFFSET k`): simple, allows page jumping, but the DB must skip k rows on every query — $O(n)$ cost that grows with depth. Also inconsistent under concurrent writes. Cursor: records the last-seen value (e.g. `createdAt + id`) and uses a `WHERE` clause. $O(\log n)$ regardless of depth; stable under writes. Cannot jump to an arbitrary page.

JUNIOR

Knows the difference; may not know the performance implications.

MID

Explains the $O(n)$ skip cost; knows cursor is sometimes called "keyset pagination".

SENIOR

Notes that the Store.API's `HasNextPage` already serves as a cursor signal; discusses composite cursor (`time + id`) to handle non-unique sort keys.

⚠ **Red flag:** Cannot name a scenario where offset pagination breaks — they've only used it, never thought about its limits.

3 What happens if the scraper inserts the same product twice? Should there be a unique constraint?

Without a constraint: duplicate rows — inflated counts, products appear twice in the UI. With `UNIQUE + UPSERT`: the second run refreshes the existing row instead of duplicating. The best column to constrain depends on what's stable. Using the scraper-generated GUID as PK doesn't help (new GUID = new row). A better approach: store the source system's integer ID as a separate unique column (`SourceId`) and upsert on that.

JUNIOR

Knows you need a unique constraint; picks Name (fragile but reasonable for demo data).

MID

Explains why GUIDs don't help; suggests storing the source ID.

SENIOR

Discusses idempotency keys, exactly-once semantics vs at-least-once, and how `UPSERT` on a natural key is the correct pattern.

4 Downsides of too many database indexes?

Each index must be maintained on every write (`INSERT`, `UPDATE`, `DELETE`) — adding latency. Indexes consume disk and buffer pool memory. The query planner can pick a suboptimal index. For the scraper (bulk inserts), dropping non-critical indexes before the run and recreating them after can dramatically speed up ingestion.

⚠ **Red flag:** Thinks more indexes are always better — has never profiled a write-heavy workload.

5 If both API access and HTML scraping are available, which do you choose?

API — always. Structured, typed data; versioned contract; no HTML parsing fragility. HTML scraping breaks silently when layout changes, is tightly coupled to presentation, and is much harder to maintain. HTML is a fallback when no API exists.

JUNIOR

Picks API because "it's easier".

MID

Explains contract stability and maintenance costs.

SENIOR

Discusses API rate limiting, robots.txt, terms of service for external sites; pagination semantics; error handling differences.

6 How would you parse a price string like "€1,299.90"?

Strip non-numeric/non-decimal characters: `price.replace(/[^\d-9.]/g, '')` → `"1299.90"`, then `parseFloat`. Validate the result is a finite positive number. Edge case: locale-dependent separators (`"1.299,90"` in Germany). Identify the locale from the page before parsing.

⚠ Red flag: Suggests `parseInt` — loses decimal places. Or uses a regex without considering locale.



7 What is the difference between `useQuery` and `useMutation`?

useQuery: reads data. Runs automatically (on mount, when stale, on window focus). Returns `isLoading`, `data`, `isError`. Cache is keyed and shared. **useMutation:** writes data (POST/PUT/DELETE). Fires only when you call `mutate()` explicitly. Provides `isPending` and callbacks (`onSuccess`, `onError`) where you typically invalidate related queries to re-fetch fresh data.

JUNIOR

"useQuery is for GET, useMutation is for POST." — acceptable.

MID

Explains the manual trigger and cache invalidation in `onSuccess`.

SENIOR

Discusses optimistic updates, rollback in `onError`, and the difference between `invalidateQueries` vs `setQueryData` for optimism.

8 What is an optimistic update?

Update the UI immediately — before the server confirms — and roll back if the request fails. In React Query: `onMutate` updates the cache, `onError` restores the previous snapshot, `onSettled` re-fetches to sync with server truth. Benefit: snappy UX. Risk: briefly showing state the server rejected.

⚠ **Red flag:** Cannot explain the rollback mechanism — they'd ship updates with no recovery path if the request fails.

9 When is Context API appropriate, and when is it not?

Good for: global or near-global state many components need — theme, auth, language, filter state across a page. **Not good for:** high-frequency updates (every keystroke triggers a re-render of all consumers), deeply nested data with complex update logic, or large apps where Zustand/Redux Toolkit gives better devtools and selector granularity. Context re-renders every consumer when the value reference changes — passing objects requires memoisation.

JUNIOR

"Context avoids prop drilling." — knows the motivation.

MID

Mentions re-render costs; knows to split contexts by update frequency.

SENIOR

Discusses `useMemo` on the value object, context selectors, and why Zustand/Jotai exist (Context was never designed as a state manager).

10 What is the difference between `GET /products/123` and `GET /products?id=123`?

`GET /products/123`: resource-oriented REST — the path identifies a single resource. Correct semantics; better CDN caching; access control per-resource; standard REST convention. `GET /products?id=123`: filtering a collection. Semantically "give me the products filtered by `id=123`". Works, but unconventional and harder to cache at the CDN layer.

⚠ **Red flag:** Cannot explain the semantic difference — treats both as equivalent HTTP calls.

11 Should filtering happen in the frontend or the backend?

Backend — almost always. The database filters millions of rows with indexes; sending all rows to the client wastes bandwidth, memory, and is unusable at scale. Frontend filtering is only appropriate when the full dataset is already in-memory and small (e.g. a dropdown of 20 categories already fetched).

JUNIOR

"Backend because it would be too slow to download everything."

MID

Mentions indexes, network cost, pagination coupling.

SENIOR

Notes the hybrid case: filter at backend, secondary client-side sort/grouping on the already-paginated page. Discusses full-text search as a backend concern (Elasticsearch, PostgreSQL FTS).

12 What validation should the API apply to page, pageSize, minPrice, maxPrice?

$\text{page} \geq 1$; $\text{pageSize} \in [1, 100]$; $\text{minPrice} \geq 0$ when present; $\text{maxPrice} \geq 0$ when present; $\text{minPrice} \leq \text{maxPrice}$ when both present. Return 400 with a descriptive message for each violation. Never silently clamp — the client must know when its input was rejected vs accepted.

⚠ Red flag: Suggests silently clamping invalid values (e.g. negative `pageSize` → 1) instead of returning 400. Clients can't distinguish a corrected response from an intentional one.

13 If the system grows from hundreds to millions of products, what changes?

Indexes on Price, CategoryName, and a full-text index on Name/Description. Switch from offset to cursor pagination. Cache categories (Redis / HTTP Cache-Control). Replace LIKE with a full-text search engine (PostgreSQL FTS, Elasticsearch). Scraper: scheduled job with retries, batch inserts, idempotency keys. Database: read replicas for API; consider partitioning by category or date. API: rate limiting, response compression, CDN for product images.

JUNIOR

Mentions indexes and maybe caching.

MID

Covers full-text search replacement, cursor pagination, and read replicas.

SENIOR

Discusses write vs read path separately; scraper as event-driven pipeline; eventual consistency trade-offs; CQRS if write load is also high.



Red Flags & Green Flags

Patterns that signal competence or concern

✓ Strong Signals

- ✓ Uses DynamicParameters (Dapper) without being prompted — never concatenates user input into SQL
- ✓ Paginates the scraper loop and breaks on HasNextPage — doesn't hardcode a page count
- ✓ Includes all filter values in useQuery's queryKey — understands cache invalidation
- ✓ Validates cross-field constraint ($\text{minPrice} \leq \text{maxPrice}$) — thinks about combinations, not just individual params
- ✓ Uses DateTime.UtcNow, not DateTime.Now — timezone-aware from habit
- ✓ Returns 400 for bad input, never 500 — clear understanding of HTTP semantics
- ✓ Adds image onError fallback — considers failure paths in UI
- ✓ Explains trade-offs when asked about design choices rather than stating one answer is always correct
- ✓ Uses enabled: !!id on useProduct — prevents unnecessary requests from undefined params
- ✓ Clears the debounce timeout on each keystroke — understands the pattern, not just copying it

● Concerning Signals

- ✗ String interpolation ``${"WHERE Name = '{search}'"}`` — SQL injection vulnerability, always probe this
- ✗ Scraper only fetches page 1 and stops — hasn't considered that data has multiple pages
- ✗ queryKey: ['products'] with no filter deps — a fundamental React Query misunderstanding
- ✗ Silent clamping instead of 400 for invalid params — clients can't distinguish rejected from accepted input
- ✗ No error states in UI — assumes the happy path always succeeds
- ✗ Cannot explain why DynamicParameters prevents injection when asked — used it but doesn't understand it
- ✗ Confuses useQuery and useMutation — has used React Query but only by copying patterns
- ✗ Suggests more indexes = always better performance — hasn't thought about write overhead
- ✗ Stores createdAt in local time — "it works on my machine" thinking
- ✗ Thinks GET /products?id=123 and GET /products/123 are equivalent — lacks REST semantics

⚠ **SQL injection is a hard stop.** If a candidate concatenates user input directly into SQL (`"WHERE Name LIKE '%" + search + "%'"`), that is a security vulnerability that should significantly affect the evaluation. Ask about it directly: "What would happen if someone passed `' ; DROP TABLE Products; --` as the search value?"



Evaluation Scorecard

Candidate: _____ | Date: _____ | Interviewer: _____

AREA	NOTES / EVIDENCE	SCORE	MAX
Scraper — basic (pagination, guid, upsert)			/ 15
Scraper — code quality (errors, logging, UTC)			/ 5
Backend — filtering + pagination (parameterized, all 4 filters, COUNT)			/ 20
Backend — validation (all params, 400 responses, cross-field)			/ 10
React Query hooks (queryKey, enabled, Axios, all 3 hooks)			/ 20
Products page (all filters wired, pagination, navigation)			/ 10
Product detail page (all fields, loading/error states, back nav)			/ 10
Code quality (naming, no injection, TS types, no dead code)			/ 5
Technical discussion (Q&A depth, explanations, trade-offs)			/ 5
Stage 4 — debounce			+5
Stage 4 — Context API integration			+5
Stage 4 — database index			+5
Total	Passing threshold ≈ 65 / 95		/ 95 + 15

✓ Strong Hire

85+ pts · All core tasks complete · No SQL injection · Solid Q&A depth · At least one bonus

● Hire with Discussion

65–84 pts · Most tasks complete · Minor gaps in validation or query · Can explain their choices

● No Hire

<65 pts · SQL injection present · No understanding of React Query cache · Cannot explain basic choices

Overall Notes
